EECS 348 Group One

C++ Mathematical Compiler

Test Cases

Revision History

Date	Version	Description	Author
05/05/2026	1.0	Initial Test Cases	Hudson
05/05/2026	1.1	More Test Cases and Sections 1-6	Logan
06/05/2025	1.2	Actual Results, Sections 6-7	Hudson

Table of Contents

1. Purpose 4

2. Test case identifier 4

3. Test item 4

4. Input specifications 4

5. Output specifications 4

6. Environmental needs 4
   - 6.1.1 Hardware 4
   - 6.1.2 Software 4
   - 6.1.3 Other 4

7. Special procedural requirements 5

8. Intercase dependencies 5

Test Case

1. Purpose

The purpose of the Test Case document is to format and test all the test cases provided to us and the ones that each member came up with. We decided that a table was the cleanest and simplest way to convey the amount of test cases we used, so below are all our test cases ordered 1 – 72 with a 47 Valid and 25 Invalid test case splits. The table also contains the results and if the solver passed or failed the test case.

Test Case ID	Description	Input Data	Expected Results	Actual Results	Pass/Fail
TC01	Simple Addition	3 + 4	7	7	Pass
TC02	Subtraction with Parentheses	8 - (5 - 2)	5	5	Pass
TC03	Multiplication and Division	10 * 2 / 5	4	4	Pass
TC04	Exponentiation	2 ** 3	8	8	Pass
TC05	Mixed Operators	4 * (3 + 2) % 7 - 1	5	5	Pass
TC06	Complex Addition with Extraneous Parenthesis	(((2 + 3))) + (((1 + 2)))	8	8	Pass
TC07	Mixed Operators with Extraneous Parenthesis	((5 * 2) - ((3 / 1) + ((4 % 3))))	6	6	Pass
TC08	Nested Parentheses with Exponents	(((2 ** (1 + 1)) + ((3 - 1) ** 2)) / ((4 / 2) % 3))	4	4	Pass
TC09	Combination of Extraneous and Necessary Parentheses	(((((5 - 3))) * (((2 + 1))) + ((2 * 3))))	12	12	Pass
TC10	Extraneous Parentheses with Division	((9 + 6)) / ((3 * 1) / (((2 + 2))) - 1)	-60	-60	Pass
TC11	Combining Unary Operators with Arithmetic Operations	+(-2) * (-3) – ((-4) / (+5))	6.8	6.8	Pass
TC12	Unary Negation and Addition in Parentheses	-(+1) + (+2)	1	1	Pass
TC13	Negation and Addition with Negated Parentheses	-(-(-3)) + (-4) + (5)	-2	-1	Pass
TC14	Unary Negation and Exponentiation	+2 ** (-3)	0.125	0.125	Pass
TC15	Combining Unary Operators with Parentheses	-(2) * (3) - (-4) / (-5)	-6.8	-6.8	Pass
TC16	Unmatched Parentheses	2 * (4 + 3 - 1	ERR	ERR	Pass
TC17	Operators Without Operands	* 5 + 2	ERR	ERR	Pass
TC18	Incorrect Operator Usage	4 / 0	ERR	ERR	Pass
TC19	Missing Operator	5 (2 + 3)	ERR	ERR	Pass
TC20	Invalid Characters	7 & 3	ERR	ERR	Pass
TC21	Mismatched Parentheses	(((3 + 4) - 2) + (1)	ERR	ERR	Pass

TC22	Invalid Operator Usage	((5 + 2) / (3 * 0))	ERR	ERR	Pass
TC23	Invalid Operator Sequence	((2 -) 1 + 3)	ERR	ERR	Pass
TC24	Missing Operand	((4 * 2) + (-))	ERR	ERR	Pass
TC25	Invalid Characters	((7 * 3) ^ 2)	ERR	ERR	Pass
TC26	Verify Associativity of Exponentiation Operator	2 ** 3 ** 2	512	512	Pass
TC27	Modulo Operator with Negated Operand	-14 % 3	-2	-2	Pass
TC28	Multiple Stacked Unary Operators	-(-(5)) + (+(-2))	3	3	Pass
TC29	Consecutive Operators without Operands	5 + * 3	ERR	ERR	Pass
TC30	Invalid Second Modulo Operand	10 % 0	ERR	ERR	Pass
TC31	Associativity of Division Operator	100 / 10 / 2	5	5	Pass
TC32	Associativity when Mixing Multiplication and Modulo	10 * 5 % 3	2	2	Pass
TC33	Stacked Unary Operators and Exponentiation	-(-2 ** 3)	8	8	Pass
TC34	Extraneous Parentheses Resulting in Skewed Tree	(1 + (2 + (3 + (4 + 5))))	15	15	Pass
TC35	Simple Operations with Excessive Whitespace	2 + 3 * 4	14	14	Pass
TC36	Simple Operations with No Whitespace	2+3*4	14	14	Pass
TC37	Parentheses with Excessive Whitespace	(2 + 3) * 4	20	20	Pass
TC38	Whitespace Between Unary Operators	- 5	-5	-5	Pass
TC39	Empty Parentheses	()	ERR	ERR	Pass
TC40	Missing Operator Between Parentheses and Operand	(3 + 4) 5	ERR	ERR	Pass
TC41	Invalid Operands	a + b	ERR	ERR	Pass
TC42	Simple Addition	5 + 3	8	8	Pass
TC43	Simple Subtraction	10 - 4	6	6	Pass
TC44	Simple Multiplication	6 * 7	42	42	Pass
TC45	Simple Division	20 / 4	5	5	Pass
TC46	Remainder Check	10 % 3	1	1	Pass
TC47	Exponent	3 ** 4	81	81	Pass
TC48	Parenthesis with Exponents	(2 + 3) * 4	20	20	Pass
TC49	Division and Multiplication	100 / (2 * 5)	10	10	Pass
TC50	Double Exponents	2 ** 3 ** 2	512	512	Pass
TC51	Simple Addition with prefix	+5 + 3	8	8	Pass

TC52	Simple Addition with Negative	-4 + 10	6	6	Pass
TC53	Addition Mixed with Parenthesis	-(3 + 2)	-5	-5	Pass
TC54	Testing Parentheses Awareness	(((7)))	7	7	Pass
TC55	Modulo and Additon	15 % 4 + 1	4	4	Pass
TC56	Addition and Multiplication	2 + 3 * 4 - 1	13	13	Pass
TC57	More With Multiplication and Addition but added Parentheses	(8 - 3) * (2 + 4)	30	30	Pass
TC58	Double Modulo	100 % 7 % 3	2	2	Pass
TC59	Another Parentheses Multiplication check	((2 + 3)) * ((4 - 1))	15	15	Pass
TC60	Testing sign Prefix	+(-5 + 15) * 2	20	20	Pass
TC61	Math inside Exponents	2 ** 2 + 3 * (10 - 4) % 5	7	7	Pass
TC62	Division by zero error	5 / 0	ERR	ERR	Pass
TC63	Missing left operand	* 3 + 2	ERR	ERR	Pass
TC64	Unmatched opening Parenthesis	(4 + 5	ERR	ERR	Pass
TC65	Unmatched closing Parenthesis	7 + 3)	ERR	ERR	Pass
TC66	Invalid character/operator	4 @ 2	ERR	ERR	Pass
TC67	Invalid operator sequence	6 + * 2	ERR	ERR	Pass
TC68	Missing operator between operands	5 3 + 2	ERR	ERR	Pass
TC69	Division by zero error	(10 / (2 - 2))	ERR	ERR	Pass
TC70	Invalid character	8 + $ 3	ERR	ERR	Pass
TC71	Missing right operand	((3 + 4) * (2 -))	ERR	ERR	Pass
TC72	Invalid operator sequence	3 ** / 2	ERR	ERR	Pass

We decided that a table would be the best way to represent all the test case data, so a lot of our sections will tell you to refer to the table to provide proof or data.

Test case identifier

All Test cases have an identifier of TC which means test case then followed by a number 01 – 72. (All in first column of table

2. Test item

Anything being tested in the problem should be mentioned in the description as long as it's not obvious. For example the problem ((2 + 3)) * ((4 - 1)) has both addition and subtraction, but the real test is checking the parenthesis priority and if it knows the basic PEMDAS rules.

3. Input specifications

All inputs and the equations are specified in the second column which provides a short description of what each of the problems are testing like addition or math inside exponents

4. Output specifications

The outputs are broken into two columns which is the expected and actual results so we can compare the answers to prove the compiler is working as intended

5. Environmental needs

5.1.1 Hardware

The hardware is very basic as any laptop or computer that can download the makefile and run it should be viable. So, any working laptop or desktop should be enough. Phones and tablets can't download the required files though.

5.1.2 Software

The software requirements are also very simple and should be able to run on most devices like a laptop and computer. Phones and tablets are not compatible for the same reasons as the hardware section. Any system should be able to run the makefile and test the solver.

5.1.3 Other

Download file if you wish to use GUI

6. Special procedural requirements

To check all of the tests, first compile the binary using `make` and the included makefile. Then, navigate to the `tests` directory and run `python3 test.py`. This will automatically run each test expression and compare it with the expected result. It will then provide a summary of the tests that passed and the tests that failed and then save the results to a new csv file. Each test case can also be tested individually by simply running the binary, entering the case being tested and viewing the output returned in the cli.

7. Intercase dependencies

All test cases listed in the table are isolated and can be executed independently.